Accuknox Assignment -2

Task-1:Rating

1. Large Language Models (LLMs) — Rating: A

I have practical experience in LLM application development through projects such as Finance GPT and AdvocadabraLLm. In Finance GPT, I used a Mistral model and applied LoRA fine tuning with a custom dataset to adapt the model to a finance-related use case. I have also authored a research paper on AdvocadabraLLm, reflecting my involvement in research and development related to LLM-based applications.

2. Deep Learning — Rating: B

I have worked on deep learning projects, including a Flood Detection system. I used a Convolutional Neural Network (CNN) to address the flood-detection problem. This project provided me with practical experience applying deep-learning techniques to a real-world problem.

3. Artificial Intelligence (AI) — Rating: A

I have developed AI based projects, including Finance GPT, AdvocadabraLLM, and an Intelligent Support Ticket Router. These projects involve applying AI techniques to different problems, including finance related LLM applications and automated support-ticket classification and routing. My project development and research experience have helped me explore practical applications of AI.

4. Machine Learning (ML) — Rating: A

I completed a Machine Learning internship at Codveda Technologies, where I trained multiple machine learning models. This experience helped me develop practical skills in working with datasets, training models, and evaluating their performance. Along with my internship, I have gained additional experience applying machine-learning techniques through my projects.

Task-2 :ChatBot Architecture

Objective:

The goal is to explain creation of Chatbot and how it generates relevant responses using a Large Language Model (LLM) with a help of E-commerce Use case.

Components of ChatBot:

1)Frontend:

- It is a User Interface between the User and the Application .It Takes Input /Query from the User and passes Requests onto the Backend to Search answer for the Query .

- It also displays chatbot responses and User's Query

2)Backend:

- It Receives User's Query and answers it with help of RAG (Retrieval Augmented Generation),Rag searches the context in the Database which stores Previous Conversations and Additional Information using Conversation_ID

-The Database Stores work policy and Additional information for the customers to answer their FAQ's)

-backend stores company's work policy and conversations with users in a Database with separate tables

-Backend gives context to LLM using Rag and also looks after the security Layer

- LLm generates Responses and sends it back to the frontend

3)Knowledge Base:

-The database consist of a separate section/table which stores additional information of company (like their work policy and Shipping information)

-the base also stores product details and answered FAQ's

4)RAG(Retrieval Augmented Generation):

-The RAG acts like a information retrieval system where it sends Context to LLM before it generates a response

-It retrieves the information which is useful rather than displaying the whole database

5)Coversation History Database

-This part is used to store conversations and retrieve information from previous conversations

-It uses Conversation_ID to continue and remember user's query (Ex query 1- my product is damaged ,query 2-when will receive the refund) like this system will reply while storing user's conversations and understand the context

-New messages are stored after each interaction

6)Database Design

-The Database consists of two tables one storing the work policy and shipping information,the second table consists of conversation's history along with their customer ID's

-Separate tables allow backend to retrieve different types of information separately

7)LLM(Large Language Model)

-The Large Language Model is used to process the prompt prepared by the backend.The LLm is integrated with Open ai Api so that it doesn't run out of things to say ,but it doesn't mean it will hallucinate and create new policies we make sure that the information is purely retrieved from Database or from the Company Website by instructing the system

-If the information is too sensitive and has higher priority it will be directly routed to Customer's Service of the Company

8)Security Layer

-This layer looks after authorization,authentication,System credentials of the customers and protects sensitive information

-It avoids sending sensitive information to external LLM services and maintains data privacy

9)Response Handling/Server Handling

Response Handling:This part is used to handle API failures,timeouts,invalid responses

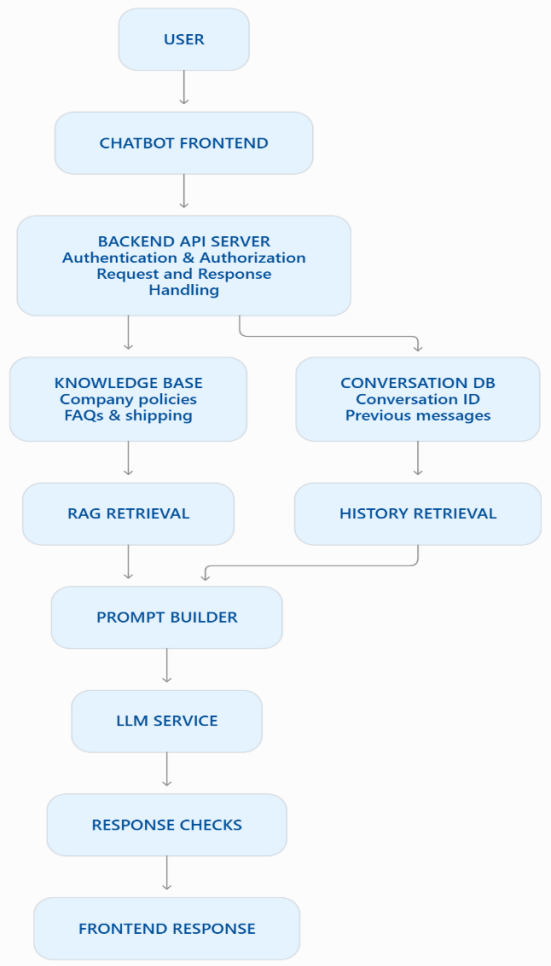
-It sends final response to frontend,also stores conversation messages in database,verifies the answer before sending it to frontend

Server Handling:This Part ensures that the server isn’t overloaded and sends requests in an order by using various techniques such as Priority queues,load balancing to distribute requests across instances

-It also scales up the resources if number of users increase

-It receives and processes incoming requests

Architecture Diagram of Chatbot Workflow



Task-3:Vector DataBase

Vector DataBase:A Vector database is a database where it stores numerical values/representations of data called vectors .The vectors are generated using embedding models which convert the text,images and other data into numerical representations.

-It is used for retrieving similar data using similarity retrieval ,these are often used in Search Engines and they find similar data using semantic score and if the score it more then the word is related or connected to the keyword.

Problem: AI-powered legal document search system for a law firm that maintains a large collection of court judgments, legal opinions, and case summaries

-The users in legal domain such as lawyers,journalists ,law students often find cases with particular keywords such as sections,jurisdiction with a particular date

-The proposed system would use embeddings to represent legal documents as vectors and retrieve semantically relevant results.

Data Assumption:

-Database Consisting of 1,00,000 legal documents

-The application developed using Python

-The System must support Similarity Search

-An open source solution which can be developed under its own infrastructure

-The system should be capable of future growth

Database For the Problem:

-For this problem I would select Qdrant because:

Metadata filtering**:** Legal documents need to be filtered by attributes such as jurisdiction and case type. Qdrant supports filtering alongside vector search which is relevant to the system.

Python integration: Qdrant provides a Python client that can integrate with your legal document processing and retrieval pipeline.

Scalability: Qdrant supports scaling deployments to accommodate growing workloads.

Self hosted Deployment:It prefers to manage its own infrastructure.

Limitations of Qdrant:

- Search results depends partly on the embedding model. Poor embeddings may fail to identify relevant legal concepts.

- A semantic similarity does not guarantee legal relevance. A document may discuss a similar topic but concern a different jurisdiction or legal issue.

-Performance depends on factors such as dataset size, hardware, indexing , filtering conditions, and query volume. Benchmarking would be necessary to determine whether the system meets its performance requirements.

Conclusion:

-For this system I would like to choose Qdrant because its vector search, metadata filtering, Python integration, and self-hosted deployment options align with the requirements.

-However I might Even consider Chroma or Milvus if the application's workload or deployment requirements change .

THANK YOU